# Báo cáo phân tích (I/O) & Thiết kế Kiến trúc Đồng bộ Mã nguồn

## Phần 1: Báo cáo phân tích (I/O) - Luồng dữ liệu khi đồng bộ Local - Cloud

Khi một lập trình viên JavaScript thực hiện thao tác lưu file mã nguồn trên máy cục bộ, luồng dữ liệu (`Input/Output`) diễn ra theo các bước chuẩn xác sau:

1. **Input (Tác vụ người dùng):**
    a. Lập trình viên chỉnh sửa file code (ví dụ: `app.js`) và nhấn tổ hợp phím **`Ctrl + S`** trên trình soạn thảo (VS Code / WebStorm).
2. **Xử lý cục bộ (Local Processing):**
    a. Hệ điều hành ghi nhận thay đổi và ghi nội dung mới trực tiếp vào ổ cứng SSD cục bộ (`D:\CampusPay\src\app.js`). Tốc độ ghi cực nhanh (~3000 MB/s).
3. **Trigger đồng bộ (Auto Sync Trigger):**
    a. Tiến trình ngầm của phần mềm Cloud (ví dụ: *Google Drive for Desktop* hoặc *OneDrive*) quét thấy sự thay đổi tệp trong thư mục được cấu hình giám sát.
4. **Lọc dữ liệu (Exclusion Filter):**
    a. Công cụ đồng bộ kiểm tra danh sách loại trừ (`.gitignore`, quy tắc đồng bộ). Nếu tệp nằm trong thư mục hợp lệ (ví dụ: thư mục `src`), tiến trình tiếp tục; nếu tệp nằm trong thư mục bị cấm (như `node_modules`), hệ thống lập tức bỏ qua.
5. **Truyền tải dữ liệu (Data Transit & Cloud Versioning):**
    a. Phần mềm đẩy bản sao tệp qua Internet lên đám mây (Cloud Storage). Hệ thống trên mây cập nhật dữ liệu mới và tạo ra mốc lịch sử phiên bản mới (**Version History**).
6. **Output (Trạng thái hoàn tất):**
    a. Quá trình tải lên kết thúc thành công. Biểu tượng trạng thái cạnh tên file trên File Explorer chuyển từ hình vòng tròn xoay sang **dấu tích xanh (`Green Checkmark`)**, xác nhận dữ liệu đã được bảo vệ an toàn trên Cloud.

## Phần 2: Thiết kế sơ đồ tổ chức thư mục Cloud chuẩn cho Lập trình viên

Để tối ưu hóa không gian lưu trữ, tận dụng tính năng **Files On-Demand** (chỉ tải file về máy khi mở) và ngăn chặn tình trạng quá tải do file rác (`node_modules`), cấu trúc thư mục Cloud chuẩn của một lập trình viên được thiết kế khoa học như sau:

Plaintext

```
📁 CampusPay-Project (Thư mục gốc đồng bộ Cloud)
 ├── 📁 01. Code/                    <-- CHỈ đồng bộ mã nguồn thuần túy
 │    ├── 📁 src/                    (Source code chính: .js, .ts, .vue)
 │    ├── 📁 public/                 (Assets tĩnh công khai)
 │    ├── 📄 package.json            (Định nghĩa thư viện - KHÔNG chứa node_modules)
 │    └── 📄 package-lock.json       (Khóa phiên bản thư viện)
 │
 ├── 📁 02. Docs/                    <-- Tài liệu đặc tả kỹ thuật
 │    ├── 📄 API_Specification.md
 │    └── 📄 Database_Schema.png
 │
 ├── 📁 03. Assets/                  <-- Tài nguyên đồ họa, UI/UX
 │    ├── 📁 ui_designs/
 │    └── 📁 marketing_banners/
 │
 └── 📄 .gitignore                   <-- Cấu hình loại trừ thư mục nặng khỏi Git/Sync
```

## Phần 3: Giải quyết bẫy dữ liệu (`node_modules`) & Files On-Demand

- **Bẫy `node_modules` và cách xử lý:**
  - **Vấn đề:** Thư mục `node_modules` trong JavaScript chứa hàng chục nghìn file nhỏ (đôi khi lên đến vài trăm MB hoặc hàng GB). Nếu đồng bộ thư mục này lên Cloud, tiến trình sẽ bị "treo" (freeze) do nghẽn băng thông xử lý tệp tin nhỏ.
  - **Giải pháp cốt lõi: Tuyệt đối không đồng bộ `node_modules`.**
    - Khai báo thư mục `node_modules/` vào tệp `.gitignore` và cấu hình ngoại lệ (Exclusion) trong phần mềm đồng bộ Cloud.

        - Khi một thành viên mới clone code về máy, họ **không** copy thư mục `node_modules` mà chỉ cần chạy lệnh **`npm install`** trong Terminal. Hệ thống sẽ tự động tải lại các thư viện cần thiết trực tiếp từ npm registry về máy cục bộ một cách nhanh chóng và an toàn.
- **Tận dụng Files On-Demand:**
    - Bật tính năng **Files On-Demand** (hoặc *Stream Files*) trên Google Drive for Desktop/OneDrive. Các tài nguyên nặng trong thư mục `Assets` hoặc `Docs` sẽ nằm trên mây ở trạng thái "chờ", chỉ khi nào lập trình viên mở file thì hệ thống mới tải xuống SSD tạm thời, giúp tiết kiệm tối đa dung lượng ổ cứng cho máy tính cá nhân.